

Abstract

This document proposes Vehicle Machine Language (VML), a machine-native behavior representation for autonomous vehicles and advanced driver assistance systems.

VML is motivated by the observation that natural language is highly effective for human communication but poorly suited for direct machine execution. Unlike natural language, VML provides explicit semantics, deterministic interpretation and formal verification capabilities.

The purpose of VML is not to replace human requirements writing. Instead, VML serves as the target representation into which human requirements are translated.

Future systems may employ Large Language Models (LLMs) or Vision-Language Models (VLMs) during requirements engineering to perform this translation. However, no runtime component of the vehicle is expected to consume natural language directly.

This document defines the first conceptual version of VML and provides sufficient detail to enable an initial implementation.

Vehicle Machine Language (VML)

Draft White Paper Version 0.1

Working Draft

July 28, 2026

Origin: <https://twg-fas.org>

1 Introduction

Modern autonomous systems process increasingly complex information.

A typical driving decision may depend on:

- observed objects,
- traffic regulations,
- temporal constraints,
- mission goals,
- safety requirements,
- uncertainty estimates.

Most existing systems represent these concepts using separate subsystems.

VML attempts to provide a common formal representation for all behavior-relevant information.

The long-term vision is:

$$\textit{Natural Language} \rightarrow \textit{VML} \rightarrow \textit{Vehicle Behavior}$$

where natural language exists exclusively during engineering and VML becomes the native runtime language.

2 Design Principles

VML is built around six principles.

2.1 P1: No Runtime Natural Language

No runtime decision shall depend on natural-language interpretation.

$$\textit{Runtime} \cap \textit{NaturalLanguage} = \emptyset$$

2.2 P2: Explicit Semantics

Every VML statement shall have one unique interpretation.

2.3 P3: Composability

Complex behavior shall be constructed from simpler behavior.

2.4 P4: Verifiability

Every VML specification shall be suitable for formal analysis.

2.5 P5: Human Readability

While machine-oriented, VML should remain understandable to engineers.

2.6 P6: Technology Independence

VML shall not depend on any specific perception system, planning algorithm or neural network architecture.

3 VML Overview

VML consists of six layers.

$$VML = L \oplus T \oplus M \oplus P \oplus G \oplus D$$

where

$$L : \textit{LogicalLayer} \tag{1}$$

$$T : \textit{TemporalLayer} \tag{2}$$

$$M : \textit{MetricLayer} \tag{3}$$

$$P : \textit{ProbabilisticLayer} \tag{4}$$

$$G : \textit{GoalLayer} \tag{5}$$

$$D : \textit{DeonticLayer} \tag{6}$$

Each layer answers a distinct question.

Layer	Question
Logic	What exists?
Temporal	When must it happen?
Metric	How much?
Probability	How certain are we?
Goal	What do we want?
Deontic	What is allowed?

4 Logical Layer

The logical layer defines facts.

Example:

`Pedestrian(P1)`

`Vehicle(V4)`

`Lane(L2)`

Interpretation:

- object P1 is a pedestrian
- object V4 is a vehicle
- object L2 is a lane

Relationships are represented as predicates.

InLane(V4, L2)
AheadOf(P1, Ego)

The logical layer provides the vocabulary used by the remaining layers.

5 Temporal Layer

The temporal layer defines behavior over time.

VML adopts four basic temporal operators.

5.1 Always

$$\Box\varphi$$

Meaning:

“ φ must always hold.”

Example:

$$\Box(\neg Collision)$$

5.2 Eventually

$$\Diamond\varphi$$

Meaning:

“ φ must eventually become true.”

Example:

$$\Diamond ParkingCompleted$$

5.3 Next

$$\bigcirc\varphi$$

Meaning:

“ φ must hold in the next state.”

5.4 Until

$$\varphi U \psi$$

Meaning:

“Maintain φ until ψ becomes true.”

Example:

$$BrakingUPedestrianClear$$

6 Metric Layer

Driving requires continuous values.

Examples include distance, velocity and acceleration.

VML introduces built-in arithmetic expressions.

Examples:

$$Distance(Ego, P1) > 2m$$

$$Velocity(Ego) < 30km/h$$

$$LateralAcceleration(Ego) < 0.3g$$

Metric expressions may be combined with temporal operators.

Example:

$$\Box(Distance(Ego, P1) > 2m)$$

7 Probabilistic Layer

Perception is uncertain.

Instead of representing facts as strictly true or false, VML associates confidence values.

Example:

$$Probability(Pedestrian(P1)) = 0.95$$

VML introduces confidence thresholds.

Example:

$$Require_{0.99}(SafeDistance)$$

Interpretation:

Safety shall hold with at least 99% confidence.

8 Goal Layer

Goals describe desired future states.

Example:

Goal `ParkingCompleted`

or

Goal `At(Ego, ChargingStation)`

Goals differ from constraints.

Constraint:

$$\Box(\neg Collision)$$

Goal:

$$\Diamond ParkingCompleted$$

A valid plan must satisfy all constraints while attempting to achieve all goals.

9 Deontic Layer

The deontic layer represents obligations, permissions and prohibitions.

These concepts map naturally to traffic regulations.

Obligation:

$$O(YieldToPedestrian)$$

Permission:

$$P(ChangeLane)$$

Prohibition:

$$F(CrossSolidLane)$$

Traffic regulations should preferably be represented using deontic constructs rather than procedural code.

10 Example

Natural language requirement:

The vehicle shall stop for pedestrians and remain stopped until the crossing is clear.

VML representation:

$$O(YieldToPedestrian)$$

$$\Box(PedestrianCrossing \rightarrow (Braking \cup CrossingClear))$$

This representation is substantially more explicit and directly machine-processable.

11 Example Runtime Architecture

A reference implementation may follow the architecture:

$$Sensors \rightarrow Perception \rightarrow VML\ State \rightarrow Planner \rightarrow Controller$$

The perception system updates predicates.

The planner searches for actions satisfying all constraints.

The controller executes the selected action.

12 VML Core Profile

The first implementable VML profile is intentionally limited.

Required features:

- predicates,
- temporal operators,
- metric expressions,
- goals,

- obligations.

Optional future features:

- probabilistic reasoning,
- utility functions,
- learning components,
- ontology inheritance,
- multi-agent reasoning.

This subset is sufficient for constructing an initial parser, runtime monitor and planner.

13 Conclusion

VML is proposed as a machine-native language for autonomous systems.

The central philosophy is simple:

$$\textit{NaturalLanguage} \neq \textit{RuntimeLanguage}$$

Human requirements are authored in natural language.

They are translated into VML.

Runtime systems operate exclusively on VML.

In this manner natural language assumes the role of source code, while VML becomes the executable behavioral representation used within the vehicle.

Future versions of this document will extend the language, formalize syntax and define reference execution semantics.